

Nx AI Manager — Stress Dashboard Post-Processor

A capacity-planning & stress-testing dashboard for the NX AI Manager (Nx Meta and Nx Witness).

Overview

The Stress Dashboard is a custom NX AI Manager post-processor that answers the question every integrator and pre-sales engineer faces: “**For this model and this many cameras, what frame rate do I actually get, and how hard is the hardware working?**” It runs as a transparent pass-through — measuring throughput without altering inference results — and pairs that with whole-machine CPU, RAM, GPU and NPU load. The result is a live web dashboard plus an exportable, per-run stress-test report.

What it solves

- **Capacity planning:** how many camera channels can a box sustain with a chosen model before FPS drops?
- **Bottleneck identification:** which resource saturates first (CPU? GPU? NPU?) — so you know what to upgrade or where to offload inference.
- **Benchmarking:** repeatable, shareable HTML/CSV reports per model + camera-count + hardware combination.

How it works

Two cooperating components, following the standard NX AI Manager external-postprocessor pattern:

- **Post-processor:** counts inference frames per camera channel (by `DeviceID`) and forwards per-second counts to the web app. It never modifies the inference results.
- **Web app:** samples whole-machine CPU/RAM/GPU/NPU once per second, turns frame counts into FPS, serves the dashboard, records named runs, and exports reports.

Data path: NX AI Manager → (Unix socket) → Post-processor → (HTTP) → Web app → SQLite → Dashboard / Report.

What it measures

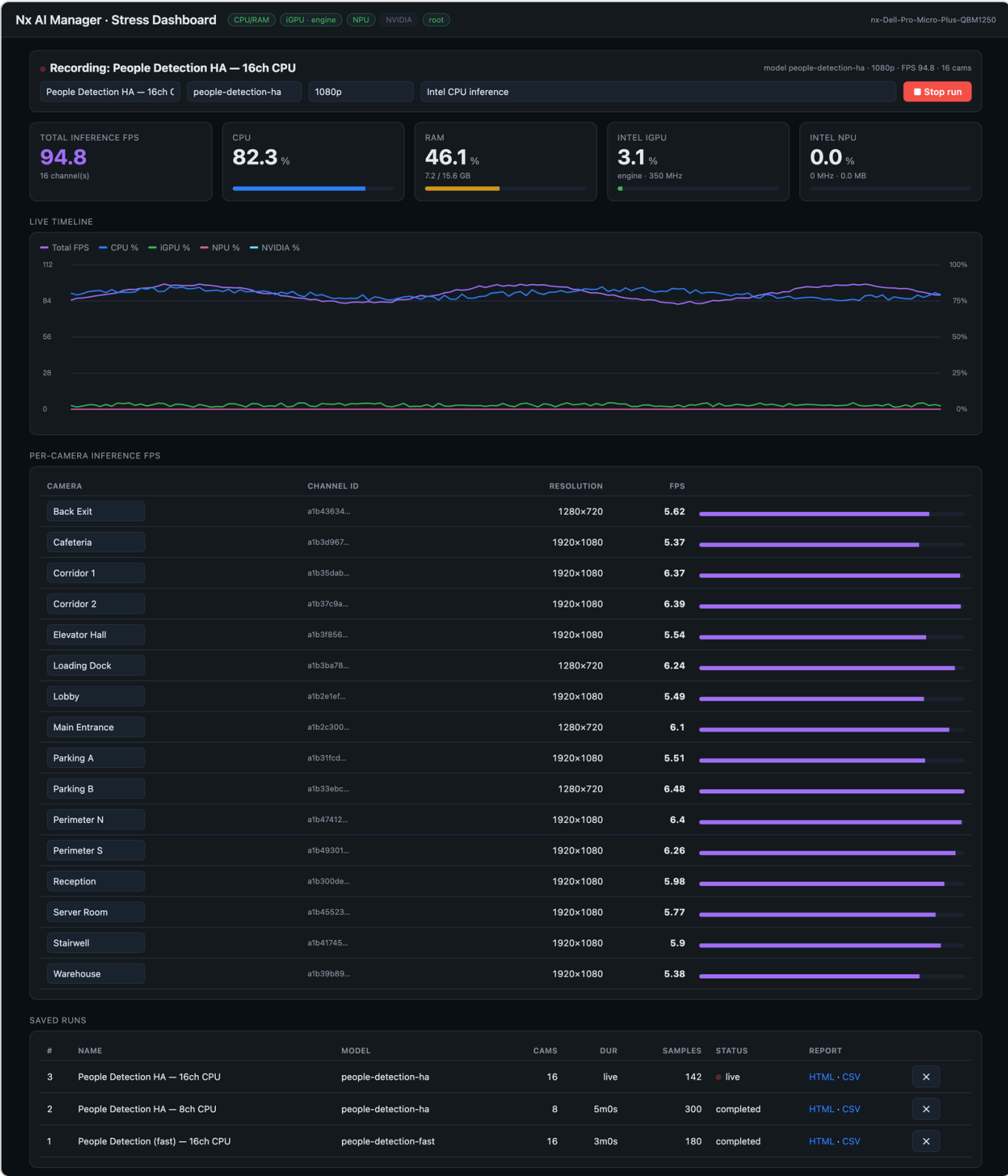
- **Live resource load** — whole-machine CPU, RAM, GPU and NPU.
- **Total throughput** — total concurrent inference FPS across every channel running this post-processor.
- **Per-channel FPS** — per-camera FPS (channels auto-discovered, renamable).
- **Stress report** — start/stop a named run, export self-contained HTML (charts + avg/peak/min/p95) and CSV.
- **Portable** — auto-detects Nx Meta and Nx Witness; metric sources auto-detect and degrade gracefully.

Metric sources (Linux)

Resource	Source
CPU / RAM	psutil (whole machine)
Intel NPU	intel_vpu sysfs — busy-time → utilisation %, memory, frequency (no root needed)
Intel iGPU	xe / i915 DRM — true engine utilisation via per-client fdinfo (root); frequency-ratio fallback otherwise
NVIDIA GPU	NVML (pynvml) — utilisation, memory, power, temperature

Live Dashboard — simulated example

Scenario: **16 cameras** analysed with the **People Detection model (high accuracy)**, inference running on the **Intel CPU** (Core Ultra 7 265).



Reading the example:

- **Total throughput:** ~94.8 FPS across all 16 channels — roughly 5.9 FPS per camera.
- **Where the load is:** CPU is the bottleneck at ~82%, while the Intel iGPU (~3%) and NPU (~0%) sit idle — exactly what you expect when inference is pinned to the CPU.
- **Takeaway:** offloading this model to the NPU or iGPU would free substantial CPU headroom and likely raise the achievable channel count — precisely the decision this tool informs.

Stress-Test Report (HTML / CSV export)

Stopping a run produces a self-contained report (opens offline, no server needed): run configuration, an avg / peak / min / p95 summary for every metric, per-camera FPS, and time-series charts.

Nx AI Manager · Stress-Test Report

Generated 2026-06-08 20:47:48

RUN CONFIGURATION

Run name	People Detection HA — 16ch CPU
Model	people-detection-ha (high accuracy)
Cameras	16
Resolution	1920x1080
Notes	Intel CPU inference (Core Ultra 7 265)
Host	nx-Dell-Pro-Micro-Plus-QBM1250
Started	2026-06-08 20:44:48
Duration	180.0 s
Samples	180

SUMMARY (AVG / PEAK / MIN / P95)

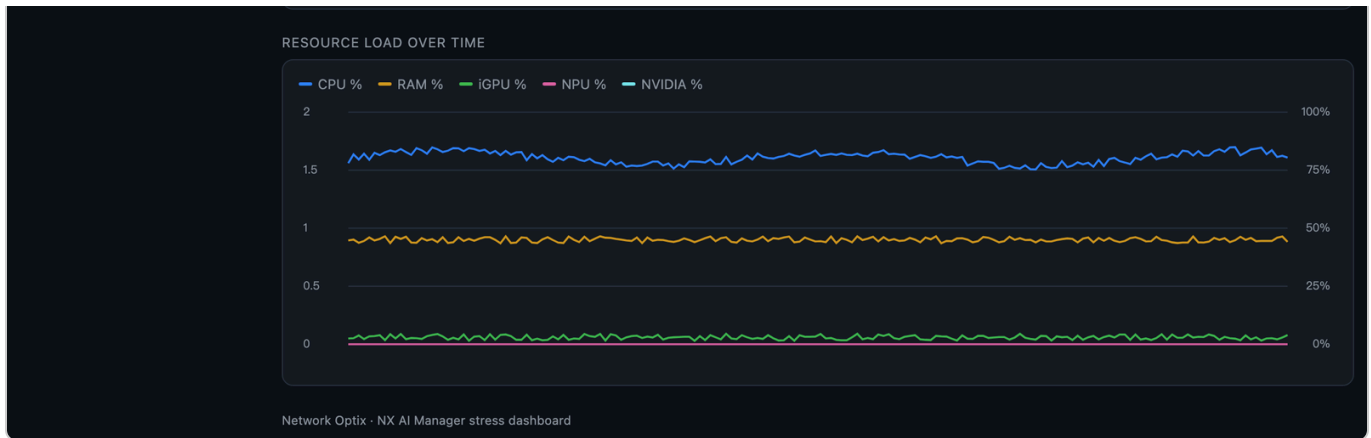
METRIC	AVG	PEAK	MIN	P95
Total inference FPS	89.39	97.74	81.26	96.48
CPU load	80.41%	84.9%	75.3%	84.1%
RAM used	7200.15 MB	7397.9 MB	7000 MB	7385.8 MB
RAM load	44.99%	46.5%	43.5%	46.4%
Intel iGPU load	2.94%	4.5%	1.5%	4.3%
Intel NPU load	0%	0%	0%	0%
Intel NPU memory	0 MB	0 MB	0 MB	0 MB

PER-CAMERA INFERENCE FPS (AVG / PEAK / MIN / P95)

CAMERA	CHANNEL ID	AVG	PEAK	MIN	P95
Main Entrance	a1b2c300-c000-e000-0000-000000000000	6.06	6.78	5.36	6.63
Lobby	a1b2e1ef-c001-e00d-001d-0000000f4243	4.93	5.68	4.22	5.57
Reception	a1b300de-c002-e01a-003a-0000001e8486	5.38	6.07	4.67	5.92
Parking A	a1b31fcd-c003-e027-0057-0000002dc6c9	5.3	6.03	4.64	5.9
Parking B	a1b33ebc-c004-e034-0074-0000003d090c	6.24	6.95	5.49	6.82
Corridor 1	a1b35dab-c005-e041-0091-0000004c4b4f	6.14	6.85	5.44	6.73
Corridor 2	a1b37c9a-c006-e04e-00ae-0000005b8d92	6.53	7.23	5.81	7.07
Warehouse	a1b39b89-c007-e05b-00cb-0000006acfd5	5.03	5.72	4.34	5.65
Loading Dock	a1b3ba78-c008-e068-00e8-0000007a1218	5.65	6.4	4.93	6.26
Cafeteria	a1b3d967-c009-e075-0105-00000089545b	4.92	5.67	4.21	5.5
Elevator Hall	a1b3f856-c00a-e082-0122-00000098969e	5.28	6	4.57	5.88
Stairwell	a1b41745-c00b-e08f-013f-000000a7d8e1	5.8	6.54	5.1	6.42
Back Exit	a1b43634-c00c-e09c-015c-000000b71b24	4.91	5.66	4.2	5.53
Server Room	a1b45523-c00d-e0a9-0179-000000c65d67	5.25	5.99	4.56	5.82
Perimeter N	a1b47412-c00e-e0b6-0196-000000d59faa	6.07	6.8	5.34	6.69
Perimeter S	a1b49301-c00f-e0c3-01b3-000000e4e1ed	5.89	6.61	5.14	6.43

INFERENCE FPS OVER TIME





Exported stress-test report for the same 16-camera CPU run (avg $\approx$ 89 FPS, peak $\approx$ 98 FPS).

Deployment

One command deploys both components (auto-detects Meta vs Witness, installs deps, registers the post-processor without clobbering others, starts the web app):

```
./deploy.sh <server_ip> <ssh_user> <ssh_password> 8120 --start
```

Open `http://<server>:8120` , enable “Stress Dashboard” on the cameras under test, and run the model.

Run the web app as root for true Intel iGPU engine utilisation; CPU, RAM and NPU work without root. GPU/NPU load reads ~0 until inference actually runs on that device.

Network Optix · NX AI Manager — Stress Dashboard post-processor